EvoRationale: A Stateful Agent for Evidence-Grounded Revalidation of Architectural Decisions during Software Evolution

Anonymous Authors

***Abstract*—Architectural decisions often remain in force after the assumptions and evidence that justified them have changed. Existing architecture-recovery, rationale-mining, and traceability techniques recover artifacts or detect structural divergence, but they do not determine whether post-decision evidence continues to support an accepted decision. We formulate *architectural decision revalidation* and *design-rationale drift*, and introduce EVORATIONALE, a bounded, stateful agent that, given an adjudicated decision state, retrieves temporally eligible artifacts, links them to rationale elements, preserves verified findings across investigation rounds, checks material claims, and recommends KEEP, REVALIDATE, REVISE, or REOPEN for human review. We evaluate EVORATIONALE against seven prespecified baselines on 1,200 held-out episodes from 40 repositories across two organizations, using repository-disjoint partitions and shared initial states, model settings, and budgets. EVORATIONALE achieves drift and outcome macro-F_1 scores of 0.82 and 0.79, improving over the prespecified comparator by 0.08 and 0.09. Evidence-retrieval and evidence-link F_1 reach 0.84 and 0.78, gains of 0.06 and 0.10, while evaluator-estimated unsupported-claim rate decreases from 0.15 to 0.07. A system-blinded audit corroborates this improvement in grounding: repository-weighted human-adjudicated UCR is 0.086 versus 0.171, a paired absolute reduction of 0.085 (95% CI [0.062, 0.110]). These gains require 22.8% more execution time and 21.9% higher API cost than the stateless tool agent. All five single-component ablations degrade their prespecified target measures, with explicit linking showing the largest conditional contribution. The findings support evidence-grounded post-decision assistance; automatic initial-state construction and practitioner effects remain unevaluated.**

***Index Terms*—architectural decisions, software evolution, design rationale, agentic AI, evidence grounding, temporal reasoning, software architecture**

I. INTRODUCTION

Architectural decisions often outlive the conditions that justified them. As systems evolve, workloads, dependencies, requirements, quality priorities, and operational evidence may change even when implementations remain structurally conformant [1]–[3]. Existing approaches address adjacent concerns: architectural-knowledge and rationale techniques capture or recover decision context [4]–[6]; architecture recovery and conformance detect implementation divergence [7], [8]; and traceability and repository-level retrieval locate related artifacts [9]–[11]. None directly reassesses whether post-acceptance evidence still justifies an accepted decision. We call this task *architectural decision revalidation* and define *design-rationale drift* as a mismatch between recorded rationale and evolved project context. Revalidation requires more than retrieval. Evidence must fall between decision acceptance and the evaluation cutoff, remain within the repository boundary, identify the evaluated artifact version, and preserve provenance. It must also be linked to the goal, assumption, requirement, constraint, quality attribute, dependency, or prior evidence that it affects. Finally, drift and action must remain distinct: incomplete or conflicting evidence may require further investigation, whereas sufficiently strong evidence may justify revising or reopening a decision. Every material claim must therefore be traceable to eligible evidence. We present EVORATIONALE, a bounded, stateful single agent that maintains a versioned decision state, derives unresolved evidence needs, retrieves and links temporally eligible artifacts, classifies their evidential roles, and recommends KEEP, REVALIDATE, REVISE, or REOPEN. Decision Memory retains verified findings, unresolved questions, rejected interpretations, and verifier feedback across bounded rounds. Deterministic checks validate identifiers, timestamps, provenance, citations, evidence relations, and output structure; Alloy contributes evidence only for formalizable constraints, and humans retain final authority. We evaluate EVORATIONALE against seven baselines on 1,200 held-out episodes from 40 repository-disjoint projects. Giving every system the same adjudicated S_0 isolates revalidation from state construction. EVORATIONALE improves all six prespecified effectiveness and grounding measures, with the human audit corroborating lower UCR, at higher execution cost. Five full-corpus ablations and an Alloy-subset analysis examine the contributions of components.

Contributions. This work (1) formulates architectural decision revalidation, six drift categories, and four actionable outcomes; (2) presents a stateful architecture combining decision memory, temporal retrieval, explicit linking, bounded investigation, and verification; (3) constructs a longitudinal, independently annotated benchmark with decision states, ordered evidence, evidence-to-rationale links, evidential relations, drift labels, and outcomes; and (4) provides a repository-disjoint evaluation of effectiveness, grounding, efficiency, cross-organizational variation, and component contributions under controlled inputs and budgets.

II. MOTIVATING EXAMPLE AND PROBLEM DEFINITION

A. Motivating Example

Consider a team that accepts a Redis-based caching decision at time t_0 to satisfy a peak-load latency target. Its rationale assumes a predominantly read-heavy workload, infrequent profile updates, and manageable cache invalidation [7], [8].

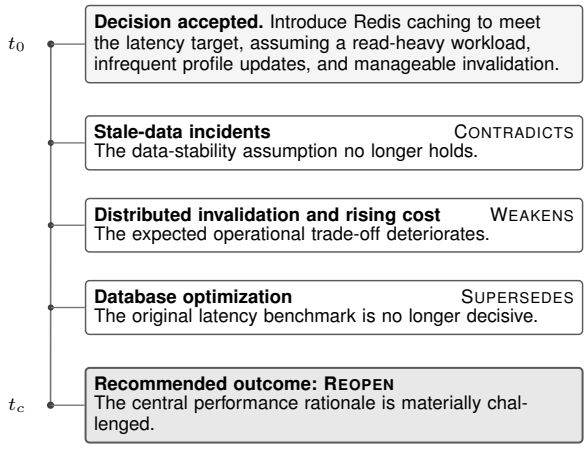


Fig. 1. Motivating timeline: post-decision evidence weakens or contradicts rationale and leads to REOPEN at t_c.

After acceptance, stale-data incidents contradict the update assumption, distributed invalidation weakens the expected operational trade-off, and database optimization supersedes the benchmark that originally justified caching (Figure 1). At cutoff t_c, the implementation may still conform to the selected design, but the combined evidence materially challenges its central justification. No individual artifact determines the outcome: the recommendation to REOPEN follows from temporally ordering the evidence and relating each artifact to the specific rationale element it affects.

B. Architectural Decision Revalidation

Let d denote an architectural decision accepted at time t_0. Its initial decision state is

$$S_0 = \langle d, G, \mathcal{R}, A, E, Q, C, I \rangle, \tag{1}$$

where G, $\mathcal{R}$, A, E, and Q represent the decision's goals, requirements, assumptions, original evidence, and affected quality attributes, respectively. C contains constraints and dependencies, while I records implementation scope and explicit validity conditions. These elements may originate from multiple project artifacts rather than a single architectural decision record [3], [4], [12]–[14]. Let $H_{t_c} = \langle h_1, \ldots, h_n \rangle$ denote the repository history ordered by the earliest verifiable timestamp of each artifact version, with $t_0 < \tau(h_i) \leq t_c$. Temporal eligibility is necessary but not sufficient for evidential use: a retained artifact must also be relevant to a state element and assigned an explicit relation from *supports*, *weakens*, *contradicts*, *supersedes*. An evidence record preserves the artifact identity, version, timestamp, provenance, affected state element, and assigned relation. Architectural decision revalidation is the mapping

$$\mathcal{F}(S_0, H_{t_c}) \mapsto O_{t_c} = \langle y, T, E^*, \mathcal{U}, r, a, \mathbf{c} \rangle, \tag{2}$$

where y is the recommended outcome; T is a multi-label set of detected drift types; E^* contains the verified evidence records; $\mathcal{U}$ contains unresolved questions; r is the evidence-grounded rationale; a is the proposed next action; and $\mathbf{c}$ contains the associated confidence scores. The multi-label set T may contain one or more of six drift types: assumption, evidence, requirement, implementation, quality-attribute, and dependency drift. The recommendation y is one of KEEP, REVALIDATE, REVISE, or REOPEN; drift type alone does not determine it. KEEP indicates that the available evidence continues to support the central rationale and reveals no material reason for change. REVALIDATE indicates that missing or conflicting evidence prevents a defensible judgment and requires further investigation. REVISE indicates that the decision remains defensible, but sufficient evidence requires modification of its scope, conditions, or recorded rationale. REOPEN indicates that the central justification is materially weakened, contradicted, or superseded and that alternatives should be reconsidered. Every outcome is a recommendation for human review; S_0 becomes a new authoritative state only after human approval.

III. RELATED WORK

A. Architectural Knowledge, Rationale, and Evolution

Architectural-decision models organize design context, alternatives, rationale, consequences, and quality trade-offs [3], [4], [15], [16], while rationale models connect decisions to requirements and design elements [17]. In practice, such knowledge is often incomplete, scattered, or lost during software evolution [5], [6], [13], [18]. Recent LLM-based approaches generate design rationale and architectural decision records (ADRs) or recover architectural documentation and views [19]–[22]. Kantara, the closest rationale-mining approach, extracts rationale from commit messages, constructs a knowledge graph, and identifies relationships and potential conflicts among historical decisions [23]. Kantara recovers and relates historical rationale; EVORATIONALE evaluates whether post-acceptance evidence continues to justify a particular decision. Architecture-recovery and erosion techniques instead detect structural divergence between intended and implemented architectures [24]–[26], including code–documentation inconsistencies and architectural-rule violations [7], [8], [27]. These techniques assess implementation alignment, whereas revalidation assesses the continuing validity of the decision's evidential basis.

B. Traceability, Retrieval, and Stateful Agents

Traceability research connects architectural decisions and components to source artifacts [9], recovers decisions from communication channels [28], and reconstructs issue-to-commit links through repository navigation [10]. Repository-level reasoning spans files, dependencies, execution behavior, and development history [11], [29]. Tool-using agents support navigation, localization, planning, and modification [30]–[33], while retrieval methods improve context selection through iterative retrieval, component-aware RAG, reranking, adaptive signals, and filtering [34]–[38]. These systems generally retrieve or connect artifacts for a current development task. EVORATIONALE instead derives evidence needs from a historical decision state, restricts retrieval to the decision's

post-acceptance interval, and assigns retained artifacts explicit evidential relations to affected rationale elements. Its memory also differs from repair-oriented experience stores such as EXPEREPAIR [39]: rather than transferring repair experiences across tasks, Decision Memory preserves verified evidence, unresolved questions, rejected interpretations, and verifier feedback for one evolving decision.

C. Verifier-Guided Reasoning

Verifier-guided agents use external feedback to test and refine generated outputs. Existing approaches incorporate proof-assistant feedback or Alloy counterexamples into iterative correction [40], [41]. The EVORATIONALE verifier addresses a different correctness boundary: it validates repository provenance, artifact versions, timestamps, temporal eligibility, citations, evidence relations, and output structure. Alloy is invoked only for formalizable decision constraints, and its instances or counterexamples are treated as evidence rather than architectural verdicts.

IV. THE EVORATIONALE APPROACH

EVORATIONALE orchestrates retrieval, linking, memory, analysis, and verification as internal tools of one bounded agent. Figure 2 shows the deployment workflow, and Algorithm 1 specifies the automatic loop from an available S_0, which is also the controlled-evaluation boundary.

A. System Boundary and Orchestration

The orchestrator converts unresolved state elements into evidence questions and a structured report. Its outer retrieve–link–verify–update–analyze–validate sequence, checkpoints, and stopping bounds are fixed rather than model-generated. Within this scaffold, the LLM selects and reformulates questions and repository queries, interprets artifacts, classifies evidential relations, and revises its investigation from verifier feedback; it is never treated as evidence. Deterministic gates enforce repository, temporal, and artifact-version boundaries and validate identifiers, provenance, citations, labels, and output structure. The agent cannot alter these boundaries, bypass verification, approve recommendations, or overwrite authoritative state. This bounded agency preserves adaptive investigation while keeping evidential admissibility and architectural authority auditable and outside model control.

B. Decision-State Construction and Memory

During deployment, the Decision-State Builder automatically proposes a draft S_0 based on ADRs, issues, pull requests, commits, and design discussions. Each candidate element retains its artifact identifier, version, source location, earliest verifiable timestamp, and extraction confidence. Explicit project statements populate the draft state; unsupported inferences are marked uncertain, and missing rationale becomes an unresolved question rather than fabricated evidence. Human confirmation makes the draft authoritative. The replay evaluation bypasses this construction step and supplies every system with the same adjudicated S_0. Decision Memory maintains the resulting working state, evidence needs, verified links, rejected interpretations, verifier feedback, and current assessment without modifying the authoritative state.

Algorithm 1 Bounded post-decision revalidation

Require: Initial state S_0, history H_{t_c}, interval $(t_0, t_c]$, budget B
Ensure: Evidence-grounded report R for human review

$M \leftarrow \text{INITIALIZEMEMORY}(S_0)$
$\mathcal{U} \leftarrow \text{FORMULATEEVIDENCENEEDS}(S_0)$
while $\mathcal{U} \neq \emptyset$ and $\neg\text{STOP}(B, M)$ **do**
  $q \leftarrow \text{SELECTQUESTION}(\mathcal{U}, M)$
  $X \leftarrow \text{RETRIEVE}(q, H_{t_c}, (t_0, t_c], B)$
  $L \leftarrow \text{LINKANDCLASSIFY}(X, S_0, q)$
  $(L^+, F_E) \leftarrow \text{VERIFYEVIDENCE}(L, X)$
  $M \leftarrow \text{UPDATEMEMORY}(M, L^+, F_E)$
  $R \leftarrow \text{ANALYZEANDPLAN}(S_0, M, \mathcal{U})$
  $(v, F_R) \leftarrow \text{VALIDATEREPORT}(R, M)$
  $M \leftarrow \text{RECORDFEEDBACK}(M, F_R)$
  $F \leftarrow F_E \cup F_R$
  $\mathcal{U} \leftarrow \text{UPDATENEEDS}(\mathcal{U}, q, L^+, F)$
  if $v = \text{PASS}$ and $\mathcal{U} = \emptyset$ **then**
    return R
  end if
end while
return $\text{UNRESOLVEDREPORT}(S_0, M, \mathcal{U})$

C. Temporal Evidence Retrieval and Linking

For each unresolved state element, the Temporal Query Planner records the target element, relevant components and artifact types, interval $(t_0, t_c]$, and stopping condition. Repository tools execute these questions iteratively and expand retrieval only when the available evidence or verifier feedback identifies a concrete gap [10], [34], [35]. A deterministic temporal gate rejects a candidate unless its exact artifact version belongs to the authorized repository and existed within the eligible interval. The Evidence Linker then records the artifact identifier, version, timestamp, source location, provenance, affected state element, and evidential relation. Only verified links enter Decision Memory. A question is resolved only when sufficient direct evidence passes verification; if repeated searches yield no new relevant artifact or the budget is exhausted, the question remains explicitly unresolved.

D. Drift Analysis, Verification, and Human Review

The Drift Analyzer compares S_0 with the verified evidence timeline. Every drift claim must identify an affected state element, a supporting artifact, and an explicit evidential relation; a drift category does not directly determine the revalidation outcome. The Action Planner synthesizes verified relations, conflicting evidence, coverage, and unresolved questions to recommend an outcome and next action. The Output Validator checks schemas, labels, identifiers, timestamps, provenance, duplicate references, and confidence ranges. The Evidence Verifier reopens the exact cited artifact versions and checks

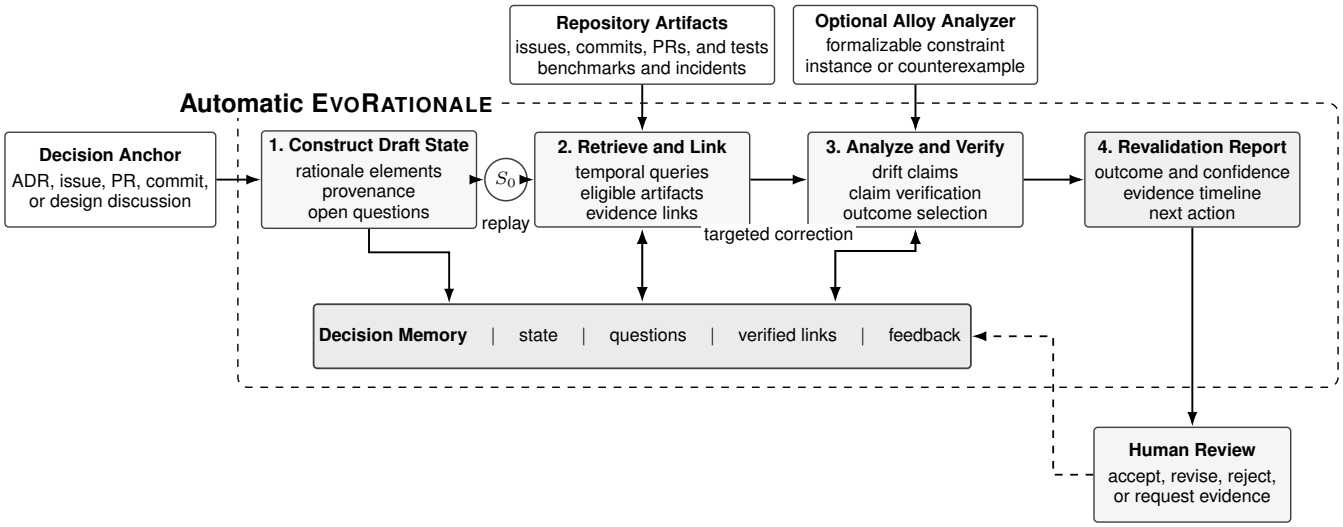


Fig. 2. Architecture of EvoRationale. Stage 1 drafts S_0, whereas replay uses the shared adjudicated S_0. Solid one-way and bidirectional arrows denote workflow and memory access; dashed arrows denote correction or feedback. Only human review updates the authoritative state.

their content, relevance, timing, and assigned relations. Failed checks become targeted evidence needs for a bounded correction round and cannot be resolved from the LLM's parametric knowledge. For formalizable constraints, Alloy may produce an instance or counterexample [41]; the result is stored with its command, scope, originating state element, and provenance and is treated as evidence rather than an architectural verdict. The final report presents the recommendation, verified evidence, affected elements, unresolved questions, confidence level, and proposed next action for human review.

V. Study Design

We evaluate post-decision revalidation through repository-disjoint retrospective replay under shared initial states, immutable snapshots, and budgets. Four RQs examine decision effectiveness, evidence grounding, efficiency, organizational variation, and component contributions.

A. Research Questions

RQ1. How accurately does EvoRationale detect design-rationale drift and predict revalidation outcomes?
RQ2. How accurately does it retrieve, link, and classify post-decision evidence, and how well does that evidence ground its reports?
RQ3. What execution overhead does EvoRationale incur relative to the strongest baseline, and how do its outcome performance and report grounding vary between the two organizations?
RQ4. How much does each architectural component contribute to effectiveness, grounding, and execution efficiency?

B. Benchmark Construction

We constructed a longitudinal benchmark of 1,500 architectural-decision episodes from 50 proprietary enterprise repositories across two organizations (Table I). Repositories, rather than individual episodes, were the partitioning unit. The organization-stratified development partition contains 300 episodes from 10 repositories, and the held-out partition contains 1,200 episodes from 40 repositories; no repository appears in both. Protocol refinement, prompt selection, and parameter tuning were used only on the development partition. Repository access was restricted to authorized study personnel under applicable data-use agreements. Reporting uses blinded Organization A/B labels and excludes repository, developer, URL, internal identifier, source excerpt, and other identifying information. An episode consists of a decision accepted at t_0, an adjudicated initial state S_0, an evaluation cutoff t_c, and the repository history in $(t_0, t_c]$. The cutoff is the timestamp of the immutable repository snapshot used for replay. It was fixed before evidence and outcome annotation and was not selected according to the presence of drift, the amount or direction of evidence, or the eventual outcome. Consequently, every evaluated configuration receives the same history ending at t_c. Decision anchors originate from ADRs, design documents, issues, pull requests, commits, or design discussions. We classify an anchor as explicit when a dedicated project record states the decision and as implicit when linked artifacts jointly establish its acceptance and scope. Both types require at least a partially documented rationale; implementation artifacts alone cannot provide an undocumented justification. Inclusion additionally requires architectural significance, an identifiable acceptance time and implementation scope, and a nonempty eligible post-decision history. Candidate pools were constructed independently of EvoRationale by combining manual repository search with the top 50 BM25 and top 50 BGE-M3 results per query, fused by reciprocal-rank fusion with $k = 60$, followed by exact-identifier and repository-link expansion. Search was capped at 12 queries per episode and stopped earlier after all direct links were inspected, and two consecutive reformulations found no new relevant parent artifact. Segments were consolidated by parent-artifact iden-

tifier and exact version; overlapping segments counted once, later edits remained distinct, and only versions first auditable in $(t_0, t_c]$ were eligible. Episode selection preceded outcome annotation and did not use revalidation labels. We excluded duplicates, routine refactorings, local implementation choices, and episodes lacking recoverable rationale or adequate history. Each retained episode contains S_0, candidate and reference evidence, evidence-to-state links and relations, drift labels, and one reference outcome. Repository-disjoint partitioning prevents project overlap, while the interval $(t_0, t_c]$ prevents pre-decision evidence leakage. During repository screening, we examined 82 repositories (43 from Organization A and 39 from Organization B). We excluded 26 because of incomplete or inaccessible snapshots (7), unverifiable decision-acceptance timestamps (6), fewer than 30 eligible episodes (9), or repository duplication or overlap (4). From the remaining 56 repositories (29/27), an organization-stratified random draw with seed 2027 selected 25 from each organization before annotation. Across these 50 repositories, 3,684 candidate episodes were assigned mutually exclusive reasons for exclusion: duplicates (286), routine refactorings (512), local implementation choices (351), unrecoverable rationale (247), or inadequate post-decision history (141), leaving 2,147 eligible episodes. Forty-four repositories exceeded the prespecified 30-episode cap, and six contained exactly 30. A repository-specific deterministic shuffle, combining seed 2027 with the repository identifier, retained 30 episodes per repository and removed 647, resulting in 1,500 episodes. Within each organization, a seed-2027 draw assigned five repositories to development and the remaining 20 to held-out evaluation before annotation. Organizational balance was imposed by design; outcome frequencies were not sampling quotas.

C. Annotation and Reference Labels

Three software architecture researchers developed the codebook using a stratified pilot of 60 development episodes, with 30 episodes from each organization. The pilot remained in the development partition and was not included in the held-out evaluation. After codebook refinement, the 60 pilot episodes were reannotated under the frozen codebook before tuning. Two researchers independently annotated every non-pilot episode, and the third adjudicated disagreements. Assignments were rotated, and annotators were blinded to system identities and outputs during initial judging. The initial candidate pools, judging rules, and residual-judging protocol were frozen before held-out execution. Artifacts retrieved outside an initial pool were not automatically scored as non-relevant: two source-system-blinded annotators judged them independently, the third adjudicated disagreements, and confirmed relevant artifacts were added to the shared reference set before final scoring of every system. Annotation proceeded in stages: candidate-evidence relevance and affected state element, evidential relation, multi-label drift categories, and finally, one revalidation outcome. Unsupported information was marked uncertain, and missing rationale remained unresolved rather than being inferred from implementation artifacts. Agreement was computed from pre-adjudication labels after the pilot and every 100 episodes using Krippendorff's α [42]. Values of $\alpha \geq 0.80$ were accepted; $0.67 \leq \alpha < 0.80$ triggered targeted review; and $\alpha < 0.67$ required codebook revision, additional training, and re-annotation. On the held-out partition, pre-adjudication α was 0.86 for evidence relevance, 0.79 for evidential relations, and 0.83 for outcomes. Binary agreement across the six drift categories ranged from 0.76 to 0.88. Because the evidence links are set-valued, we also report a pairwise F_1 score of 0.82 and a Jaccard similarity of 0.71. Relations and drift categories below 0.80 underwent targeted review; no held-out α fell below 0.67. Adjudication retained the original labels, cited evidence, final judgment, and rationale. Finally, the full team audited a stratified sample of 150 episodes—30 development and 120 held-out episodes, or 10% of the benchmark—selected by organization, anchor type, drift category, and reference outcome. Pilot episodes were excluded from this audit.

Grounding-evaluator audit. Before inspecting system or evaluator outputs, we selected 50 held-out episodes covering all 40 repositories (25 per organization). With seed 2027, SHA-256 ranking chose the lowest-ranked episode per repository and the lexicographically first feasible ten-episode extension satisfying reference-outcome (KEEP/REVALIDATE/REVISE/REOPEN= $14/11/10/15$), anchor-type (explicit/implicit= $29/21$), and coverage of all six drift categories; categories could co-occur. A second hash selected the same value across 10 repetitions for both systems, yielding 100 reports. Human segmentation removed 11 nonmaterial claims, added 7 missing claims, and split 8 compound claims, increasing the number of extracted claims from 360 to 364. After a 20-claim development calibration, two system-blinded annotators independently assessed claim support against exactly cited artifact versions, and a third adjudicated disagreements. Initial agreement ($\alpha = 0.528$) triggered codebook revision, another 20-claim calibration, and full reannotation; final agreement reached $\alpha = 0.815$ with 95.9% exact agreement. The evaluator remained frozen.

D. Baselines and Ablations

We compare seven baselines: four controlled configurations, one paper-guided reimplementation, and two task adaptations (Table II). All systems receive the same adjudicated S_0, repository snapshot, boundary $(t_0, t_c]$, generator model, inference settings, output schema, context limit, and episode budget; tool-using systems also share query and unique-artifact-read limits. They differ in evidence retrieval, organization, linking, analysis, and verification. No adaptation receives EVORATIONALE's Decision Memory or verifier.

Kantara-Reimplemented. Because the released materials lack a pinned end-to-end workflow, we follow the paper and available artifacts [23]. The reimplementation processes all temporally eligible, nonempty human-authored commit-message sentences after deterministic boilerplate and duplicate removal, without relevance or label filtering. It retains decision/rationale extraction, the Rationale and Decision

TABLE I
BENCHMARK COMPOSITION; PARENTHESIZED COUNTS SHOW THE A/B SPLIT. OUTCOMES AND ANCHORS ARE EXCLUSIVE; DRIFT LABELS ARE NOT.

Corpus and Partitions		Held-Out Outcomes and Anchors		Held-Out Drift and Evidence	
Characteristic	**Value**	**Characteristic**	**Episodes / Value**	**Characteristic**	**Episodes / Value**
Total repositories	50	KEEP	338	Assumption drift	420
Organization A / B repositories	25 / 25	REVALIDATE	271	Evidence drift	455
Development repositories	10 (5 / 5)	REVISE	249	Requirement drift	315
Held-out repositories	40 (20 / 20)	REOPEN	342	Implementation drift	370
Development episodes	300 (150 / 150)	Explicit-anchor episodes	690	Quality-attribute drift	340
Held-out episodes	1,200 (600 / 600)	Implicit-anchor episodes	510	Dependency drift	395
Total episodes	1,500	Formalizable episodes / repositories	186 / 31	Median reference evidence per episode	8
Pilot episodes	60	Reference-label audit episodes	150	Median post-decision history, months	18

Graph, `has_rationale`, *Similar*/*Contradicts* relations, and structure-based conflict detection. Adaptations add only the shared boundaries, the S_0 mapping, the provenance/version fields, and the report schema. Published examples were reproduced structurally, but not with exact probabilities or aggregate results; two researchers inspected 20 development episodes, followed by a third researcher's review.

LinkAnchor-Adapted. We validated the executable components of the official `fse26` implementation before adapting them through a private evaluation fork [10], retaining dynamic commit-chain search, lazy tools, aggregate resolution reasoning, `Store`/`Discard` feedback, context pruning, Git/diff and issue functions, and Tree-sitter navigation. An adapter maps S_0 to the issue interface and restricts retrieval to $(t_0, t_c]$; eligible unique commits in the explored chain form the evidence set, and changed components/files plus shared identifiers or rationale determine artifact–element links. Only the interface, model, and budgets are replaced.

ExpeRepair-Adapted. A private fork of upstream `main` replaces the Test and Patch Agents with Evidence Investigation and Revalidation Reasoning while retaining dynamic prompt composition [39]. Frozen episodic and semantic memories are generated without substantive manual editing from system trajectories for all 300 development episodes, including the 60 pilot episodes; BM25 retrieves the top three episodic demonstrations. The memories contain no adjudicated evidence, reference outcomes, gold drift labels, gold evidence links, or held-out information. Links are produced only through the shared output prompt, without an adapter or separate linker.

Selection protocol. Each system receives 24 candidates evaluated on the same 60 pilot episodes reannotated under the frozen codebook, yielding $1{,}440$ configuration–episode cases per system without held-out use. Selection maximizes the prespecified unweighted mean of drift macro-F_1, outcome macro-F_1, evidence F_1, evidence-link F_1, relation macro-F_1, and $1-\mathrm{UCR}$. Ties favor a lower UCR, a higher evidence-link F_1, lower cost, lower time, and a smaller configuration ID. Each case permits an initial attempt and two transient-failure retries; unrecovered cases receive zero on all five F_1 measures and $\mathrm{UCR}=1$, and candidates with more than 5% failures are ineligible. Configurations and primary comparators are frozen before held-out execution: the stateless tool agent for drift, outcome, relation, and UCR, and LinkAnchor-Adapted for evidence and linking, preventing post-hoc selection.

For RQ4, five one-component ablations remove Decision Memory, iterative retrieval, runtime verification, the lower temporal bound t_0, or explicit evidence linking. All retain the complete system's S_0, model settings, snapshot, cutoff t_c, schema, context limit, and budget, and receive no reference evidence or labels. *Without Temporal Filter* may admit versions available at or before t_0 but preserves t_c, testing historical inadmissibility rather than post-cutoff or label leakage. Their primary metrics are outcome macro-F_1, evidence F_1, UCR, drift macro-F_1, and evidence-link F_1, respectively. The Alloy analysis, also fixed before held-out inspection, compares outcome macro-F_1 and UCR on formalizable episodes.

E. Implementation and Experimental Setup

Evaluation boundary and representation. Replay operated on immutable repository snapshots under the shared inputs and budgets defined above. The deployment path normalizes authenticated GitHub and Jira webhooks into versioned Kafka events, but live ingestion, event relevance, automatic decision matching, network delay, and concurrency were outside the evaluation [43]–[47]. MongoDB stored versioned states, evidence, verifier feedback, and run metadata, while Elasticsearch indexed artifacts and extracted architecture facts. Except for the *Without Temporal Filter* ablation, an artifact version was eligible only if its first auditable availability timestamp lay in $(t_0, t_c]$. We used the introducing commit for commits and file versions; the creation or edit timestamp for issue, pull-request, review, and comment versions; and the first timestamp at which a test, benchmark, or incident record was stored in an authorized source. Embedded execution or occurrence times were retained as event metadata but could not backdate eligibility. Later edits formed new versions. The Java and Spring Boot prototype uses Spoon for program structure and framework components and dedicated parsers for manifests, configurations, deployment descriptors, API specifications, and database migrations [48]. Artifacts were segmented at semantic boundaries; units over 1,024 tokens were divided into segments of at most 1,024 tokens with a 128-token overlap, while retaining parent identifiers and provenance for artifact-level consolidation.

Retrieval and model configuration. Artifact embeddings used pinned BAAI/bge-m3 model and tokenizer revisions, 1,024-

TABLE II
BASELINE CONFIGURATIONS. LINKS DENOTE A METHOD-SPECIFIC EXPLICIT LINKER BEYOND THE SHARED OUTPUT PROMPT; PARTIAL DENOTES RELATIONAL REPRESENTATION, AND REIMPL. DENOTES A PAPER-GUIDED REIMPLEMENTATION.

System	Type	Post-decision evidence strategy	State/ Memory	Links	Verifier
Decision-only LLM	Controlled	Receives no post-decision repository evidence.	None	No	No
Long-context LLM	Controlled	Receives eligible artifacts that fit within one context window.	None	No	No
Hybrid-RAG	Controlled	Combines lexical and dense retrieval through reciprocal-rank fusion and one-shot generation [11], [35].	None	No	No
Kantara-Reimplemented	Reimpl.	Extracts rationale and relations from commit messages using a paper-guided reimplementation [23].	Rationale graph	Partial	No
LinkAnchor-Adapted	Adapted	Retains lazy, dynamic commit-chain search and maps eligible explored-chain commits to explicit decision-element links [10].	Execution-local	Yes	No
ExpeRepair-Adapted	Adapted	Retrieves from frozen episodic and semantic memories constructed on development episodes [39].	Dual memory	No	No
Stateless tool agent	Controlled	Uses the shared repository tools and may reformulate queries within one execution.	None	No	No
EVORATIONALE	Proposed	Uses temporal retrieval, iterative investigation, explicit linking, and verifier-guided correction.	Decision-centered	Yes	Yes

dimensional normalized vectors, and an 8,192-token encoder limit [49]. Each query retrieved at most 50 BM25 and 50 dense candidates, fused by reciprocal-rank fusion with $k = 60$ [50], [51]. A development-selected frozen reranking policy used component overlap, repository links, artifact type, and consistency with the target decision element. The long-context baseline performed no query-based retrieval: eligible segments were ordered in reverse chronological order, ties were resolved by artifact type and stable identifier, and whole segments were packed deterministically within the shared input budget; segments exceeding the remaining budget were omitted rather than truncated. All LLM-based configurations used `gpt-5.6-terra` through the OpenAI Responses API with `reasoning.effort=medium`, `text.verbosity=low`, `max_output_tokens=4096`, and a cumulative 64,000-token input budget per episode [52], [53]. Each episode allowed one initial round and at most two correction rounds, for at most three retrieve–link–verify rounds, twelve repository queries, and twenty-four unique artifact reads. Execution stopped after verified evidence sufficiency, repeated searches with no new relevant artifact, or budget exhaustion; unresolved needs remained explicit.

Verification and Alloy. Before a memory update, the Evidence Verifier reopened each linked artifact version and checked repository scope, temporal eligibility, provenance, content relevance, affected state element, and evidential relation; only passing links entered Decision Memory. After analysis, the Output Validator checked the report schema, labels, identifiers, citations, confidence values, and support for material claims. Failed checks produced explicit feedback and consumed the remaining correction budget. For formalizable constraints, Alloy Analyzer 6.2.0 recorded the originating state element, constraint, command, scope, result, counterexample, and provenance [54], [55]. Alloy output was treated as evidence rather than a revalidation verdict, and human review and authoritative state updates remained outside replay.

Controls and reproducibility. All parameters were selected on development repositories and frozen before held-out execution. Ten runs per episode–configuration pair logged prompts, retrieval settings, tool interactions, cited artifact versions, tokens, latency, retries, verifier results, and cost; requests excluded complete proprietary files, credentials, internal URLs, and unrelated content. A planned anonymous artifact will include code, prompts, configurations, the codebook, 24 structurally faithful synthetic episodes with inputs, evidence, links, relations, and labels, pseudonymized repository-level scores, de-identified run-level outputs, and analysis scripts; confidentiality precludes replay on real evidence. Experiments ran sequentially on Ubuntu 24.04 on a 32-core server with 256 GB of memory and an RTX 6000 Ada GPU.

F. Evaluation Metrics

Runs and aggregation. We evaluate 1,200 held-out episodes at the decision, artifact, link, and report-claim levels. The complete system and five full-corpus ablations produced $6 \times 1{,}200 \times 10 = 72{,}000$ runs, and the seven baselines produced $7 \times 1{,}200 \times 10 = 84{,}000$ runs. Disabling Alloy added $186 \times 10 = 1{,}860$ runs; the corresponding complete-system runs were reused, yielding 157,860 held-out evaluation runs; development tuning cases and retry attempts are excluded from this count. Metrics intended to characterize one execution are computed separately for each repetition. For each repository, we compute the metric over its episodes within each repetition, then average the 10 repetition-specific scores; the reported score is the unweighted mean of the 40 repository scores. Thus, repetitions quantify stochastic variation rather than form a ten-run ensemble. The Alloy analysis applies the same procedure to 186 episodes from 31 repositories. Modal outcomes are used only for recommendation-change counts: an ablation counts as changed when its modal label differs from the complete system's modal label, and the rate is divided by 1,200. A frequency tie is resolved in favor of the tied label

with the highest mean confidence among repetitions predicting that label; no tie occurred.

Drift and outcome prediction (RQ1). Drift detection is a six-label task evaluated primarily by macro-F_1, with micro-F_1, Jaccard similarity, exact-set accuracy, and per-category F_1 as secondary measures [56]. Outcome prediction is a four-class task evaluated primarily by macro-F_1, with balanced accuracy, accuracy, and per-class F_1 as secondary measures. Macro-F_1 averages the fixed label sets defined in Section II-B, and zero denominators are assigned zero. For EVORATIONALE and the prespecified stateless-tool comparator, outcome calibration uses each run's four-class probability vector to compute the multiclass Brier score $\sum_{c=1}^{4}(p_c - y_c)^2$ and top-label expected calibration error (ECE) with 15 fixed equal-width bins [57]. Unrecovered runs remain in both measures with the uniform vector $(0.25, 0.25, 0.25, 0.25)$. Both measures are descriptive and pool run–episode predictions using inverse-repository-size weights so that repositories contribute equally.

Evidence retrieval and linking (RQ2). For run r of episode j, let E_j^{ref} be the final nonempty shared artifact set after adjudication and residual judging, $\widehat{E}_{jr}$ the retained artifact set, and $x_{jr} = |\widehat{E}_{jr} \cap E_j^{\mathrm{ref}}|$. Artifact-level scores are

$$P_{E,jr} = \frac{x_{jr}}{|\widehat{E}_{jr}|}, \quad R_{E,jr} = \frac{x_{jr}}{|E_j^{\mathrm{ref}}|}, \quad F_{1,E,jr} = \frac{2P_{E,jr}R_{E,jr}}{P_{E,jr} + R_{E,jr}} \tag{3}$$

All three scores are set to zero when $\widehat{E}_{jr} = \varnothing$; F_1 is also zero when $P + R = 0$. Because scores follow the aggregation order above, aggregate F_1 is not recomputed from aggregate precision and recall. We also report Precision@10, Recall@10, and nDCG@10 with binary relevance after consolidating segments from the same parent artifact [58]. Link scoring requires an exact artifact–decision-element pair, whereas relation scoring requires an exact artifact–element–relation triple; an incorrect element or relation contributes one false positive and the corresponding false negative. The relation macro-F_1 averages the four fixed relation classes. Before verifier correction, temporal validity is the proportion of proposed links whose exact artifact version belongs to the target repository and first became auditable in $(t_0, t_c]$; provenance completeness is the proportion containing the required identifier, type, location, revision, and timestamp. Citation accuracy is the proportion of report citations whose content, provenance, and interpretation are verified, and evidence coverage is the proportion of adjudicated outcome-relevant artifacts validly cited.

Grounding. Material claims comprise evidence relations, drift claims, and factual assertions. An offline evaluator, isolated from runtime verifier outputs, marks a claim as supported only when it is grounded by an eligible cited artifact whose verified content and relation entail the claim, a reproducible Alloy result, or retained provenance that directly restates S_0; S_0 cannot support post-decision or drift claims. For every report q,

$$\mathrm{UCR}_q = \frac{N_{\mathrm{unsupported},q}}{N_{\mathrm{material},q}}, \tag{4}$$

where lower values indicate stronger grounding. The output schema requires at least one material claim; a zero-claim or invalid report is assigned UCR$= 1$ and counted separately as a report failure. UCR is computed per run and follows the aggregation order above rather than pooling claims. The system-blinded grounding audit above provides human validation of the frozen evaluator; full-corpus UCR remains evaluator-estimated. Outcome correctness is evaluated separately under RQ1.

Efficiency and ablations (RQ3–RQ4). Efficiency is measured by model invocations, wall-clock time, and estimated API cost for one episode-level run. The ten values for each episode–configuration pair are averaged; reported time and cost therefore describe one execution, not the sum of repetitions. Ablations are reported as absolute changes from the complete system in their prespecified primary measures and as descriptive changes in the remaining measures. Alloy is analyzed only on formalizable episodes.

Statistical analysis. The repository is the unit of inference. We compute 95% percentile intervals from 10,000 repository-level bootstrap resamples using seed 2027. Full-corpus resamples draw 40 repositories with replacement and retain all of their episodes; Alloy resamples the 31 eligible repositories. Paired intervals use identical sampled repository indices and the empirical 2.5th and 97.5th percentiles of the paired differences. Two-sided Wilcoxon signed-rank tests compare 40 paired repository scores, or 31 for Alloy. The grounding audit applies the same paired bootstrap to its 40 unrounded repository-score pairs; it adds no hypothesis test. Pratt's method retains zero differences during ranking but excludes them from the signed-rank sums, and ties receive average ranks [59]. We report $r_{\mathrm{rb}} = (W^+ - W^-)/(W^+ + W^-)$ [60], orienting positive values toward EVORATIONALE in baseline comparisons and the complete system in ablations; lower-is-better measures are reversed. Holm correction controls the familywise error rate at 0.05 within the prespecified RQ1 (two tests), RQ2 (four tests), and RQ4 (seven tests) families; RQ3 adds no tests. Although the Alloy analysis was fixed before held-out inspection, no separate correction family was specified; its two tests therefore join the five full-corpus ablation tests in the RQ4 family [61]. We interpret adjusted p-values together with confidence intervals, effect sizes, and absolute differences.

VI. RESULTS

Unless stated otherwise, metrics, confidence intervals, and tests follow the repository-level procedure in Section V-F.

A. RQ1: Drift Detection and Revalidation Outcomes

Table III reports the primary RQ1 results. EVORATIONALE obtains drift macro-$F_1 = 0.82$ and outcome macro-$F_1 = 0.79$. Relative to the prespecified stateless-tool comparator, the paired absolute improvements are 0.08 for drift (95% CI [0.06, 0.10]; $p_{\mathrm{Holm}} = 0.004$; $r_{\mathrm{rb}} = 0.54$) and 0.09 for outcome prediction ([0.07, 0.11]; $p_{\mathrm{Holm}} = 0.002$; $r_{\mathrm{rb}} = 0.59$). Secondary drift metrics are micro-$F_1 = 0.84$, Jaccard similarity $= 0.76$, and exact-set accuracy $= 0.64$; outcome accuracy and balanced accuracy are 0.81 and 0.79. For outcome calibration, Brier score/ECE are 0.24/0.06 for EVORATIONALE versus

TABLE III
REPOSITORY-WEIGHTED HELD-OUT RQ1 RESULTS; BOLD AND UNDERLINED VALUES DENOTE THE BEST AND SECOND-BEST, RESPECTIVELY.

System	Drift macro-F_1	Outcome macro-F_1
Decision-only LLM	0.42	0.39
Long-context LLM	0.58	0.55
Hybrid-RAG	0.66	0.63
Kantara-Reimplemented	0.56	0.52
LinkAnchor-Adapted	0.69	0.64
ExpeRepair-Adapted	0.72	0.68
Stateless tool agent	<u>0.74</u>	<u>0.70</u>
EVORATIONALE	**0.82**	**0.79**

TABLE IV
LABEL-LEVEL RQ1 PERFORMANCE OF EVORATIONALE.

Drift type	F_1	Outcome	F_1
Assumption	0.85	KEEP	0.83
Evidence	0.84	REVALIDATE	0.74
Requirement	0.78	REVISE	0.72
Implementation	0.81	REOPEN	0.86
Quality attribute	0.79		
Dependency	0.83		
Macro average	**0.82**	**Macro average**	**0.79**

0.36/0.11 for the stateless comparator; no hypothesis test was performed. Table IV reports label-level performance. Assumption and evidence drift achieve the highest F_1 scores, whereas requirement and quality-attribute drift are less accurately detected. Among outcomes, KEEP and REOPEN perform best. The confusion matrix shows that REVALIDATE and REVISE are most frequently confused when the evidence establishes an outdated or incomplete rationale but does not clearly distinguish between further investigation and an immediate modification.

B. RQ2: Evidence Retrieval, Linking, and Grounding

Table V reports the primary RQ2 measures. For evidence retrieval and linking, the prespecified comparator is LinkAnchor-Adapted. EVORATIONALE improves evidence F_1 from 0.78 to 0.84 (paired $\Delta = 0.06$; 95% CI [0.04, 0.08]; $p_{\mathrm{Holm}} = 0.009$; $r_{\mathrm{rb}} = 0.46$) and evidence-link F_1 from 0.68 to 0.78 ($\Delta = 0.10$; [0.08, 0.12]; $p_{\mathrm{Holm}} = 0.003$; $r_{\mathrm{rb}} = 0.57$). For relation classification and grounding, the prespecified comparator is the stateless tool agent. Relation macro-F_1 increases from 0.67 to 0.76 ($\Delta = 0.09$; [0.07, 0.11]; $p_{\mathrm{Holm}} = 0.006$; $r_{\mathrm{rb}} = 0.55$), while evaluator-estimated UCR decreases from 0.15 to 0.07, an absolute reduction of 0.08 and a relative reduction of 53.3% (95% CI for the paired reduction [0.06, 0.10]; $p_{\mathrm{Holm}} = 0.001$; $r_{\mathrm{rb}} = 0.62$). At rank cutoff 10, EVORATIONALE obtains Precision@10 $= 0.83$, Recall@10 $= 0.88$, and nDCG@10 $= 0.86$, versus 0.76, 0.81, and 0.79 for LinkAnchor-Adapted. Temporal validity, provenance completeness, citation accuracy, and evidence coverage are 0.97, 0.99, 0.95, and 0.96. The system-blinded audit corroborates UCR: repository-weighted evaluator estimates are 0.075 and 0.158, and human-adjudicated values are 0.086 and 0.171, for EVORATIONALE and the stateless agent, respectively. The paired human reduction is 0.085 (95% repository-bootstrap CI [0.062, 0.110]), or 49.9%. Across 364 claims, evaluator balanced accuracy is 0.892, UNSUPPORTED-class F_1 is 0.837, and evaluator–human $\kappa = 0.815$. Errors primarily involve unrelated component mentions, derivative artifacts, and indirect changes in quality priority.

C. RQ3: Execution Overhead and Cross-Organizational Variation

Table VI reports the mean held-out resources per episode-level run. Wall-clock time spans the first agent operation through the validated report, including held-out retrieval, model and verifier calls, corrections, retries, and local processing. Cost applies the August 7, 2026, prices to recorded held-out tokens [52]. Both measures exclude one-time ingestion, indexing, and embedding generation; all configuration-selection executions and retries on development data; the one-time construction of ExpeRepair-Adapted's frozen episodic and semantic memories from 300 development episodes; and hardware, storage, and human review. Accordingly, RQ3 compares steady-state held-out execution rather than total lifecycle cost. At \$0.184 and 103.9 seconds per run, the projected monthly cost and sequential time for revalidating D decisions m times are $C_{\mathrm{month}} = 0.184Dm$ USD and $T_{\mathrm{seq}} = 103.9Dm$ seconds; for $D = 100$ and $m = 1$, these equal \$18.40 and 2.89 sequential hours. This projection excludes lifecycle costs and does not measure deployment throughput.

Outcome macro-F_1 is 0.80 (95% CI [0.77, 0.83]) in Organization A and 0.77 ([0.74, 0.80]) in Organization B; evaluator-estimated UCR is 0.06 ([0.04, 0.08]) and 0.08 ([0.06, 0.10]), respectively. The absolute gaps are 0.03 and 0.02. These descriptive estimates show no large observed divergence but do not establish equivalence.

D. RQ4: Component Contributions

Table VII summarizes the five full-corpus effects. Every repository-bootstrap interval excludes zero, and all five effects remain significant under the seven-test RQ4 Holm correction ($p_H \le 0.018$); explicit linking produces the largest change.

On 186 formalizable episodes from 31 repositories, disabling Alloy reduces outcome macro-F_1 from 0.81 to 0.78 ($\Delta = -0.03$, 95% CI $[-0.05, -0.01]$; raw $p = 0.018$; $p_{\mathrm{Holm}} = 0.036$; $r_{\mathrm{rb}} = 0.36$), remaining significant under the seven-test RQ4 correction. Its evaluator-estimated UCR increase from 0.06 to 0.07 is not significant ($\Delta = +0.01$, $[-0.003, +0.023]$; raw $p = 0.110$; $p_{\mathrm{Holm}} = 0.110$; $r_{\mathrm{rb}} = 0.22$). Subset effects are not directly comparable with full-corpus ablations.

VII. DISCUSSION

Interpretation. The contribution is not a new retrieval, memory, or verification primitive, but their decision-centered composition under explicit temporal and evidential constraints. The ablation pattern suggests complementary roles: memory preserves unresolved context across rounds, whereas explicit links

TABLE V

REPOSITORY-WEIGHTED HELD-OUT RQ2 RESULTS. AGGREGATE F_1 IS NOT RECOMPUTED FROM AGGREGATE PRECISION AND RECALL; UCR IS EVALUATOR-ESTIMATED, AND LOWER IS BETTER. BOLD AND UNDERLINED VALUES DENOTE THE BEST AND SECOND-BEST RESULTS, RESPECTIVELY.

System	Evidence Retrieval			Evidence Linking			Relation Classification			UCR
	P	R	F_1	P	R	F_1	P	R	F_1	
Decision-only LLM	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.61
Long-context LLM	0.46	0.66	0.54	0.37	0.43	0.40	0.41	0.45	0.43	0.31
Hybrid-RAG	0.74	0.70	0.72	0.57	0.53	0.55	0.61	0.57	0.59	0.22
Kantara-Reimplemented	0.51	0.37	0.43	0.44	0.35	0.39	0.52	0.45	0.48	0.34
LinkAnchor-Adapted	0.75	<u>0.81</u>	<u>0.78</u>	0.65	<u>0.72</u>	<u>0.68</u>	0.66	0.62	0.64	0.19
ExpeRepair-Adapted	0.76	0.70	0.73	0.66	0.60	0.63	0.68	0.64	0.66	0.17
Stateless tool agent	<u>0.78</u>	0.74	0.76	<u>0.67</u>	0.63	0.65	<u>0.69</u>	<u>0.65</u>	<u>0.67</u>	<u>0.15</u>
EVORATIONALE	**0.86**	**0.82**	**0.84**	**0.81**	**0.75**	**0.78**	**0.78**	**0.74**	**0.76**	**0.07**

and runtime verification constrain which artifacts may justify each claim. The recurring confusion between REVALIDATE and REVISE further shows that successful retrieval does not eliminate architectural judgment when evidence is incomplete or supports modification without invalidating a decision's central rationale.

Operational use. The measured overhead favors event-triggered use after incidents, dependency or security changes, regressions, or requirement revisions rather than continuous reanalysis. Reports should therefore expose cited evidence, provenance, uncertainty, and unresolved questions, while architects retain authority over stakeholder and organizational trade-offs. Decision records that preserve assumptions, constraints, evidence, alternatives, and validity conditions provide stronger inputs for such revalidation [3], [15].

Contribution boundary. The controlled replay establishes comparative task accuracy, grounding, and per-run cost; it does not establish practitioner usefulness, architectural decision quality, or time savings.

TABLE VI

MEAN HELD-OUT RESOURCES PER EPISODE-LEVEL RUN; LOWER IS BETTER.

System	Calls	Time (s)	Cost ($)
Decision-only LLM	1.0	12.4	0.021
Long-context LLM	1.0	41.2	0.082
Hybrid-RAG	1.0	48.7	0.094
Kantara-Reimplemented	1.4	36.1	0.061
LinkAnchor-Adapted	2.9	62.4	0.118
ExpeRepair-Adapted	2.6	75.8	0.139
Stateless tool agent	3.1	84.6	0.151
EVORATIONALE	3.6	103.9	0.184

TABLE VII

FULL-CORPUS ABLATIONS (Δ = ABLATION MINUS COMPLETE); UCR IS EVALUATOR-ESTIMATED, p_H IS HOLM-ADJUSTED OVER ALL SEVEN RQ4 TESTS, AND $r_{\mathrm{rb}} > 0$ FAVORS COMPLETE.

Removed	Target	Δ [95% CI]	p_H	r_{rb}
Decision Memory	Outcome F_1	-0.07 $[-0.09, -0.05]$	0.018	0.48
Iterative retrieval	Evidence F_1	-0.08 $[-0.10, -0.06]$	0.007	0.52
Runtime verification	UCR	$+0.09$ $[+0.07, +0.11]$	0.002	0.61
Temporal filtering	Drift F_1	-0.06 $[-0.08, -0.04]$	0.010	0.50
Explicit linking	Link F_1	-0.14 $[-0.17, -0.11]$	< 0.001	0.68

VIII. THREATS TO VALIDITY

Construct validity. Evidence relevance, drift, outcomes, claim segmentation, and support remain interpretive despite frozen codebooks, independent annotation, adjudication, and agreement assessment [42]. The 50-episode audit shows strong but imperfect evaluator–human agreement; full-corpus and ablation UCR depend on generalization beyond audited reports. Reference pools may omit artifacts, and adjudicated S_0 excludes state construction and practitioner utility.

Internal validity. Timestamp error, leakage, hindsight, undocumented context, and model variation may affect replay. Repository-disjoint partitions, immutable snapshots, fixed cutoffs and configurations, and the exclusion of reference labels from system inputs mitigate, but cannot eliminate, these risks.

Comparison validity. No prior system directly implements revalidation; Kantara is reimplemented, while LinkAnchor and ExpeRepair are adapted. Section V-D records the mechanisms retained and replaced. Shared inputs, model settings, budgets, tuning, and frozen held-out configurations reduce implementation bias. Source-paper scores are not compared across different tasks, datasets, and models. The stateless tool agent is the comparator for drift, outcome, relation, and UCR; LinkAnchor-Adapted is the comparator for evidence and linking. Results apply only to these configurations and do not establish inferiority on native tasks. Missing Kantara components, domain shift, and unretained revisions limit fidelity and reconstruction.

Statistical conclusion validity. Repository-level inference and Holm adjustment address clustering and multiplicity, but 40 repositories, or 31 for Alloy, limit power. Secondary measures are descriptive. Combinatorial removals were not evaluated; single-component ablations estimate conditional removal effects and neither identify component interactions nor justify additive attribution.

External validity. Two proprietary organizations and a Java/Spring prototype limit generalization. Balanced sampling and the 30-episode cap preclude prevalence estimates; broader and live settings require evaluation.

Reliability and reproducibility. Confidentiality prevents release of real episodes, evidence, and claim-level audit records, precluding independent evidence-level replay. Synthetic cases and de-identified outputs support pipeline and statistical repro-

duction but not benchmark representativeness; hosted-model updates and unretained baseline revisions preclude identical reruns.

IX. Conclusion

Architectural decisions can remain structurally implemented even after the evidence supporting them has changed. This work formulates architectural decision revalidation and presents EvoRationale, a bounded, stateful agent that connects temporally admissible repository evidence to the rationale elements it affects. In repository-disjoint replay, EvoRationale improved all six prespecified effectiveness and grounding measures, while the human audit corroborated its lower unsupported-claim rate. The ablations indicate that decision memory, iterative retrieval, temporal filtering, explicit linking, and runtime verification each contribute conditionally within the evaluated configuration; the measured overhead favors selective rather than continuous use. These findings support evidence-grounded decision assistance, not autonomous architectural governance. Future work should evaluate automatic initial-state construction, practitioner outcomes, broader technology settings, live operation, and full-lifecycle cost.

Data availability. An anonymous replication artifact is under preparation; real study data remain confidential.

References

[1] M. M. Lehman, "Programs, life cycles, and laws of software evolution," *Proceedings of the IEEE*, vol. 68, no. 9, pp. 1060–1076, 1980.

[2] D. E. Perry and A. L. Wolf, "Foundations for the study of software architecture," *ACM SIGSOFT Software Engineering Notes*, vol. 17, no. 4, pp. 40–52, 1992.

[3] A. Jansen and J. Bosch, "Software architecture as a set of architectural design decisions," in *Proceedings of the 5th Working IEEE/IFIP Conference on Software Architecture*. IEEE, 2005, pp. 109–120.

[4] A. Tang, M. A. Babar, I. Gorton, and J. Han, "A survey of architecture design rationale," *Journal of Systems and Software*, vol. 79, no. 12, pp. 1792–1804, 2006.

[5] M. O. Demir, O. Chouseinoglou, and A. Kolukısa Tarhan, "Factors affecting architectural decision-making process and challenges in software projects: An industrial survey," *Journal of Software: Evolution and Process*, vol. 36, no. 10, p. e2703, 2024.

[6] E. G. de Santana Junior, G. B. Figueiredo, M. Peixoto, F. Araujo, C. Prazeres, I. Machado, P. A. d. M. Silveira Neto, and E. Almeida, "Architecture decision records: Adoption, impact, and developer engagement in open-source software," in *2026 IEEE 23rd International Conference on Software Architecture (ICSA)*. IEEE, 2026, pp. 335–345.

[7] A. Bucaioni, A. Di Salle, L. Iovino, L. Mariani, and P. Pelliccione, "Continuous conformance of software architectures," in *Proceedings of the 21st IEEE International Conference on Software Architecture*, ser. ICSA 2024. IEEE, 2024, pp. 112–122.

[8] R. Su, A. Bakhtin, N. Ahmad, M. Esposito, V. Lenarduzzi, and D. Taibi, "Evaluating large language models for detecting architectural design rules violations," in *2026 IEEE 23rd International Conference on Software Architecture (ICSA)*. IEEE, 2026, preprint available as arXiv:2602.07609.

[9] D. Fuchß, H. Liu, T. Hey, J. Keim, and A. Koziolek, "Enabling architecture traceability by LLM-based architecture component name extraction," in *2025 IEEE 22nd International Conference on Software Architecture (ICSA)*. IEEE, 2025, pp. 1–12.

[10] A. Akhavan, A. Hoseinpour, A. Heydarnoori, H. Bagheri, and M. Keshani, "LinkAnchor: An autonomous LLM-based agent for issue-to-commit link recovery," *Proceedings of the ACM on Software Engineering*, vol. 3, no. FSE, pp. 4184–4207, 2026.

[11] Y. Wang, S. Wang, Y. Wang, B. Zhang, D. Guo, J. Chen, and Z. Zheng, "RepoReasoner: Evaluating repository-level code reasoning ability of long-context language models," *Proceedings of the ACM on Software Engineering*, vol. 3, no. FSE, pp. 2790–2812, 2026.

[12] K. Borowa, R. Lewanczyk, K. Stpiczyńska, P. Stradomski, and A. Zalewski, "What rationales drive architectural decisions? An empirical inquiry," in *Software Architecture: 17th European Conference, ECSA 2023, Istanbul, Turkey, September 18–22, 2023, Proceedings*, ser. Lecture Notes in Computer Science, vol. 14212. Cham: Springer, 2023, pp. 303–318.

[13] B. Ahmeti, M. Linder, R. Groner, and R. Wohlrab, "Architecture decision records in practice: An action research study," in *Software Architecture: 18th European Conference, ECSA 2024, Luxembourg City, Luxembourg, September 2–6, 2024, Proceedings*, ser. Lecture Notes in Computer Science, vol. 14889. Cham: Springer, 2024, pp. 333–349.

[14] J. Keim and A. Kaplan, "From scattered to structured: A vision for automating architectural knowledge management," arXiv preprint arXiv:2601.19548, 2026.

[15] J. Tyree and A. Akerman, "Architecture decisions: Demystifying architecture," *IEEE Software*, vol. 22, no. 2, pp. 19–27, 2005.

[16] U. van Heesch, P. Avgeriou, and R. Hilliard, "A documentation framework for architecture decisions," *Journal of Systems and Software*, vol. 85, no. 4, pp. 795–820, 2012.

[17] A. Tang, Y. Jin, and J. Han, "A rationale-based architecture model for design traceability and reasoning," *Journal of Systems and Software*, vol. 80, no. 6, pp. 918–934, 2007.

[18] D. Tofan, M. Galster, and P. Avgeriou, "Reducing architectural knowledge vaporization by applying the repertory grid technique," in *Software Architecture: 5th European Conference, ECSA 2011*, ser. Lecture Notes in Computer Science, vol. 6903. Springer, 2011, pp. 244–251.

[19] X. Zhou, R. Li, P. Liang, B. Zhang, M. Shahin, Z. Li, and C. Yang, "Using LLMs in generating design rationale for software architecture decisions," *ACM Transactions on Software Engineering and Methodology*, vol. 35, no. 8, pp. 1–38, 2026.

[20] A. Gupta, R. Dhar, D. Feitosa, and K. Vaidhyanathan, "Context matters: Evaluating context strategies for automated ADR generation using LLMs," in *Proceedings of the 30th International Conference on Evaluation and Assessment in Software Engineering (EASE 2026)*. ACM, 2026, preprint available as arXiv:2604.03826.

[21] M. De Luca, T. Santilli, D. Amalfitano, A. R. Fasolino, and P. Pelliccione, "CIAO—code in architecture out: Automated architecture documentation with large language models," in *2026 IEEE 23rd International Conference on Software Architecture (ICSA)*. IEEE, 2026, preprint available as arXiv:2604.08293.

[22] S. Miryala, R. Dhar, and K. Vaidhyanathan, "LLM-based automated architecture view generation: Where are we now?" in *2026 IEEE 23rd International Conference on Software Architecture (ICSA)*. IEEE, 2026, preprint available as arXiv:2603.21178.

[23] M. Dhaouadi, B. Oakes, and M. Famelis, "Automated extraction and analysis of developer's rationale in open source software," *Proceedings of the ACM on Software Engineering*, vol. 2, no. FSE, pp. 2548–2570, 2025.

[24] L. de Silva and D. Balasubramaniam, "Controlling software architecture erosion: A survey," *Journal of Systems and Software*, vol. 85, no. 1, pp. 132–151, 2012.

[25] R. Li, P. Liang, M. Soliman, and P. Avgeriou, "Understanding software architecture erosion: A systematic mapping study," *Journal of Software: Evolution and Process*, vol. 34, no. 3, p. e2423, 2022.

[26] Y. Zhang, C. Liu, Y. Sun, Z. Xu, W. Sun, W. Li, W. Jin, and Y. Liu, "Semantic-enhanced automatic refinement of architecture recovery results using LLMs," in *Proceedings of the 48th IEEE/ACM International Conference on Software Engineering*. ACM, 2026.

[27] T. Kiecker, J. A. Sparka, M. Reuter, A. Ziegler, and L. Grunske, "CASCADE: Detecting inconsistencies between code and documentation with automatic test generation," *Proceedings of the ACM on Software Engineering*, vol. 3, no. FSE, pp. 3816–3838, 2026.

[28] M. Soliman, "Exploring architectural design decisions in mailing lists and their traceability to issue trackers," in *Software Architecture: 18th European Conference, ECSA 2024*, ser. Lecture Notes in Computer Science, vol. 14889. Springer, 2024, pp. 307–323.

[29] C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, and K. Narasimhan, "SWE-bench: Can language models resolve real-world GitHub issues?" in *Proceedings of the International Conference on Learning Representations*, 2024. [Online]. Available: https://proceedings.iclr.cc/paper_files/paper/2024/hash/edac78c3e300629acfe6cbe9ca88fb84-Abstract-Conference.html

[30] J. Yang, C. E. Jimenez, A. Wettig, K. Lieret, S. Yao, K. Narasimhan, and O. Press, "SWE-agent: Agent-computer interfaces enable automated software engineering," in *Advances in Neural Information Processing Systems*, vol. 37, 2024.

[31] Y. Zhang, H. Ruan, Z. Fan, and A. Roychoudhury, "AutoCodeRover: Autonomous program improvement," in *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis*. ACM, 2024, pp. 1592–1604.

[32] R. Bairi, A. Sonwane, A. Kanade, V. D. C., A. Iyer, S. Parthasarathy, S. K. Rajamani, B. Ashok, and S. Shet, "CodePlan: Repository-level coding using LLMs and planning," *Proceedings of the ACM on Software Engineering*, vol. 1, no. FSE, pp. 675–698, 2024.

[33] Z. Chen, R. Tang, G. Deng, F. Wu, J. Wu, Z. Jiang, V. Prasanna, A. Cohan, and X. Wang, "LocAgent: Graph-guided LLM agents for code localization," in *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics*. Association for Computational Linguistics, 2025, pp. 8697–8727.

[34] F. Zhang, B. Chen, Y. Zhang, J. Keung, J. Liu, D. Zan, Y. Mao, J.-G. Lou, and W. Chen, "RepoCoder: Repository-level code completion through iterative retrieval and generation," in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, 2023, pp. 2471–2484.

[35] Q. Ke, Y. Zhao, H. Leng, S. Zhao, and H. Wang, "Not all RAGs are created equal: A component-wise empirical study for software engineering tasks," *Proceedings of the ACM on Software Engineering*, vol. 3, no. FSE, pp. 4161–4183, 2026.

[36] S. Zhang, Y. Ding, S. Lian, S. Song, and H. Li, "CodeRAG: Finding relevant and necessary knowledge for retrieval-augmented repository-level code completion," in *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, 2025, pp. 23 278–23 288.

[37] C. Shi, M. Gao, and Z. Gao, "AIRCoder: Adaptive integration of multi-dimensional retrieval for repository-level code completion," in *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics*. Association for Computational Linguistics, 2026, pp. 25 458–25 470.

[38] Y. Huo, K. Zeng, S. Zhang, Y. Lu, C. Yang, Y. Guo, and X. Tang, "RepoShapley: Shapley-enhanced context filtering for repository-level code completion," in *Findings of the Association for Computational Linguistics: ACL 2026*. Association for Computational Linguistics, 2026, pp. 10 390–10 412.

[39] F. Mu, J. Wang, L. Shi, S. Wang, S. Li, and Q. Wang, "ExpeRepair: Dual-memory enhanced LLM-based repository-level program repair," *Proceedings of the ACM on Software Engineering*, vol. 3, no. FSE, pp. 3954–3976, 2026.

[40] H. Tu, H. Zhao, Y. Song, M. Zafar, R. Meng, and A. Roychoudhury, "Agentic verification of software systems," *Proceedings of the ACM on Software Engineering*, vol. 3, no. FSE, pp. 3558–3581, 2026.

[41] M. Alhanahnah, M. R. Hasan, L. Xu, and H. Bagheri, "An empirical evaluation of pre-trained large language models for repairing declarative formal specifications," *Empirical Software Engineering*, vol. 30, no. 5, p. 149, 2025.

[42] K. Krippendorff, *Content Analysis: An Introduction to Its Methodology*, 4th ed. Thousand Oaks, California: SAGE Publications, 2018.

[43] GitHub, "Validating webhook deliveries," https://docs.github.com/en/webhooks/using-webhooks/validating-webhook-deliveries, 2026, accessed: 2026-08-07.

[44] ——, "Troubleshooting webhooks," https://docs.github.com/en/webhooks/testing-and-troubleshooting-webhooks/troubleshooting-webhooks, 2026, accessed: 2026-08-07.

[45] Atlassian, "The Jira Cloud Platform REST API: Webhooks," https://developer.atlassian.com/cloud/jira/platform/rest/v3/api-group-webhooks/, 2026, accessed: 2026-08-07.

[46] J. Kreps, N. Narkhede, and J. Rao, "Kafka: A distributed messaging system for log processing," in *Proceedings of the 6th International Workshop on Networking Meets Databases*, 2011, pp. 1–7.

[47] Apache Software Foundation, "Apache Kafka design," https://kafka.apache.org/43/design/, 2026, accessed: 2026-08-07.

[48] R. Pawlak, M. Monperrus, N. Petitprez, C. Noguera, and L. Seinturier, "SPOON: A Library for Implementing Analyses and Transformations of Java Source Code," *Software: Practice and Experience*, vol. 46, no. 9, pp. 1155–1179, 2016.

[49] J. Chen, S. Xiao, P. Zhang, K. Luo, D. Lian, and Z. Liu, "M3-Embedding: Multi-linguality, multi-functionality, multi-granularity text embeddings through self-knowledge distillation," *arXiv preprint arXiv:2402.03216*, 2024.

[50] G. V. Cormack, C. L. A. Clarke, and S. Büttcher, "Reciprocal rank fusion outperforms Condorcet and individual rank learning methods," in *Proceedings of the 32nd International ACM SIGIR Conference on Research and Development in Information Retrieval*, 2009, pp. 758–759.

[51] Elastic, "Hybrid search," https://www.elastic.co/docs/solutions/search/hybrid-search, 2026, accessed: 2026-08-07.

[52] OpenAI, "GPT-5.6 Terra Model — OpenAI API," https://developers.openai.com/api/docs/models/gpt-5.6-terra, 2026, accessed: 2026-08-07.

[53] ——, "Model guidance: Using GPT-5.6," https://developers.openai.com/api/docs/guides/latest-model, 2026, accessed: 2026-08-07.

[54] D. Jackson, *Software Abstractions: Logic, Language, and Analysis*, revised ed. Cambridge, Massachusetts: MIT Press, 2012.

[55] AlloyTools, "Alloy analyzer 6.2.0," https://alloytools.org/download.html, 2026, accessed: 2026-08-07.

[56] M. Sokolova and G. Lapalme, "A systematic analysis of performance measures for classification tasks," *Information Processing & Management*, vol. 45, no. 4, pp. 427–437, 2009.

[57] C. Guo, G. Pleiss, Y. Sun, and K. Q. Weinberger, "On calibration of modern neural networks," in *Proceedings of the 34th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 70. PMLR, 2017, pp. 1321–1330. [Online]. Available: https://proceedings.mlr.press/v70/guo17a.html

[58] K. Järvelin and J. Kekäläinen, "Cumulated gain-based evaluation of IR techniques," *ACM Transactions on Information Systems*, vol. 20, no. 4, pp. 422–446, 2002.

[59] J. W. Pratt, "Remarks on zeros and ties in the Wilcoxon signed rank procedures," *Journal of the American Statistical Association*, vol. 54, no. 287, pp. 655–667, 1959.

[60] D. S. Kerby, "The simple difference formula: An approach to teaching nonparametric correlation," *Comprehensive Psychology*, vol. 3, p. 11.IT.3.1, 2014.

[61] S. Holm, "A simple sequentially rejective multiple test procedure," *Scandinavian Journal of Statistics*, vol. 6, no. 2, pp. 65–70, 1979.